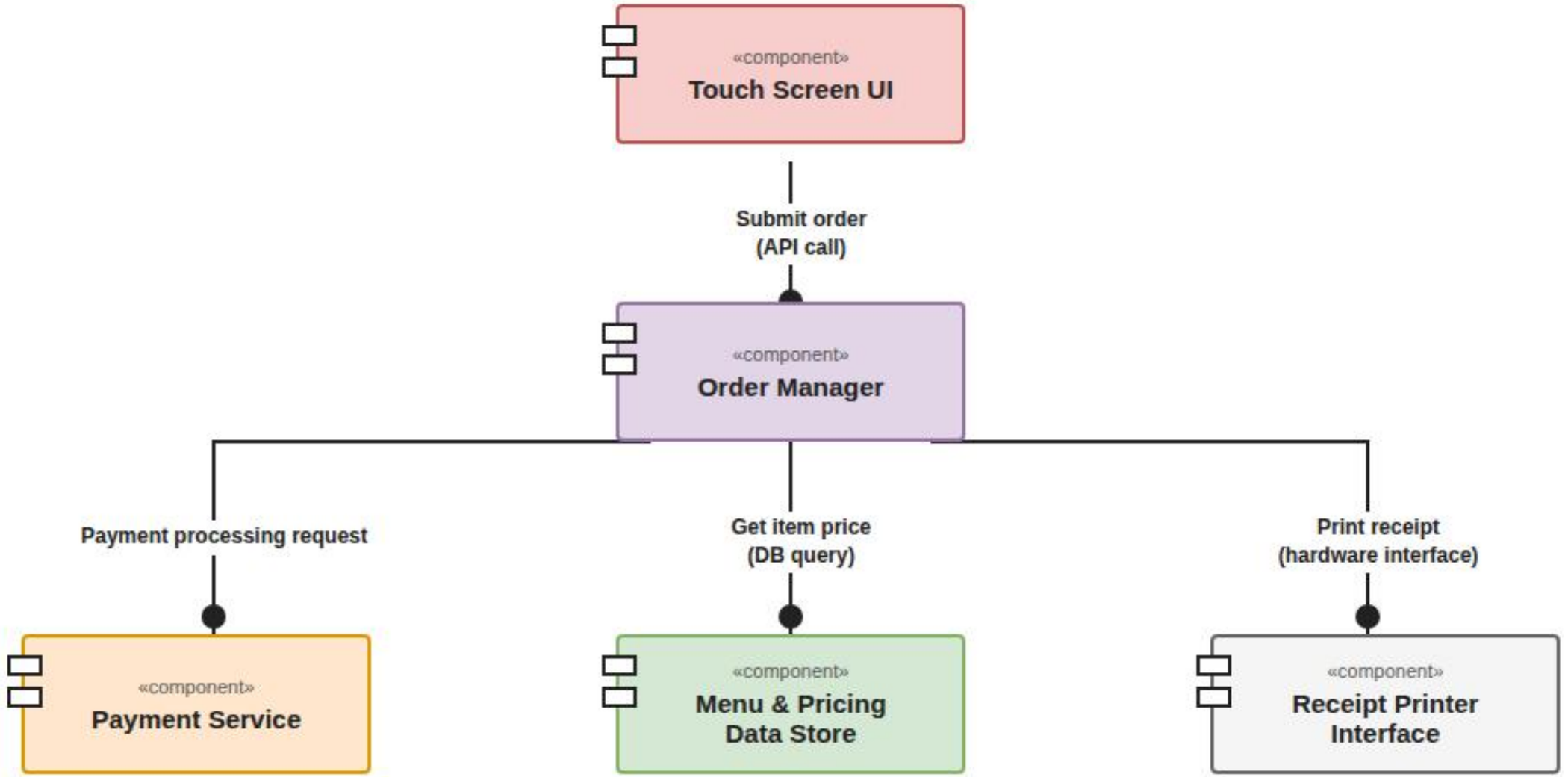

«component»
Touch Screen UI
Submit order
(API call)
«component»
Order Manager
Payment processing request
Get item price
(DB query)
Print receipt
(hardware interface)
«component»
Payment Service
«component»
Menu & Pricing
Data Store
«component»
Receipt Printer
Interface

Lab 3: Architectural Justification

Self-Service Coffee Kiosk System

We chose Client-Server Architecture for the Self-Service Coffee Kiosk System.

Architectural Choice

The kiosk is a single, self-contained device that must coordinate a small set of well-defined responsibilities: taking touchscreen input, checking menu and pricing data, processing a card payment, and printing a receipt. A Client-Server model organizes these responsibilities around a central coordinating component (the Order Manager) that requests services from clearly bounded providers (Payment Service, Menu & Pricing Data Store, Receipt Printer Interface), which matches the scenario far more directly than a Layered or Microservices approach.

Two Reasons

1. Centralized control fits a single physical device: All customer interaction happens on one kiosk, so there is no need for the independent scaling or distributed deployment that Microservices is designed for. A central Order Manager coordinating requests to a small number of well-defined services keeps the design simple and easy to reason about.

2. Clear component boundaries around distinct concerns: Payment processing, receipt printing, and menu data each have very different failure modes and update cycles (e.g., PCI compliance changes for payment vs. seasonal menu updates). A Client-Server split keeps these as separate, independently maintainable components without the operational overhead of running them as fully independent deployed services.

Security Advantage

Isolating the Payment Service as its own component behind a required/provided interface means card data and payment logic never need to pass through or be exposed to the Touch Screen UI or Menu & Pricing Data Store. The Order Manager only forwards a payment request and receives an approval/decline result, reducing the surface area that must be PCI-compliant and limiting the impact of a compromise in any other component.

Performance Benefit

Because the Menu & Pricing Data Store is a dedicated component queried directly by the Order Manager (and optionally the UI), pricing and item lookups stay local to the kiosk rather than depending on external network calls, keeping order entry fast and responsive even during busy café periods. The centralized server-side coordination also avoids the network latency and cross-service call overhead that a Microservices architecture would introduce for what is fundamentally a single-device workflow.